TEST-DAY CARD - Programming Test, 60 min

Mid-Term Programming Assessment (Week 6). Section 1 = the instructor's rules (also for the December final); sections 2-7 = my recommendations.

1. The rules (from the instructor)

Topic	Rule
Time	Click "Start attempt" -> **one hour** to do the tasks **and upload the .py file**
Late	Upload **well before** the timer ends. Late penalty = **two minor bands** (A3 -> A5, B2 -> C1)
Tasks	A series of Python tasks, **very similar to the Python task of the 2024/25 lab exam**: Task 1 and 2 need no PyTorch; **Task 3 needs PyTorch**
Output	ONE `.py` file with all tasks, **clear comments showing where each task is**
File name	`student_name_python_task.py`, e.g. `mary_ellen_foster_python_task.py` -> use YOUR name in that form
Jupyter	If you use a notebook: **File -> Save and export as -> Executable script**. Submitting `.ipynb` is **penalised**
Allowed	Course Moodle page and its materials; paper notes (handwritten / printed). **Nothing else, no other websites**
Tools	VSCode (**no Copilot / AI tools**), Spyder or Jupyter (Anaconda). No other tools. Safe Exam Browser NOT used
PyTorch	Set up your PyTorch environment **before** clicking "Start attempt" (the clock has not started). VSCode/Spyder users: **cut and paste the PyTorch solution into the .py file before submitting**

2. Before you click "Start attempt" (timer not running)

- [ ] Switch OFF every AI helper (Copilot, any chat panel or extension). Rule: no AI tools.
- [ ] In **each** environment run `import pandas, matplotlib, sqlite3, torch` and `print(torch.__version__)`. Know where torch works (Jupyter, per Moodle).
- [ ] Pick your workflow (section 4); open an empty file/notebook in a test folder; the data CSV must sit **next to** it.
- [ ] Notes on the desk: this card, Cheat Sheet v2, T9, the new T4-T7 pages, your handwritten sheet. Moodle open in one tab.

3. The 60-minute plan (marks shown are the 2024 paper's, as a guide)

Minute	Do	Why
0-3	Read ALL tasks. Underline exact names (CSV, DB, table, columns, plot titles). Look at the data: `shape`, `head()`, `isna().sum()`, `duplicated().sum()`, `dtypes`	wrong names cost marks
3-22	**Task 1** (pandas + plots). Run after each sub-task	about 13 marks
22-40	**Task 2** (SQLite). Run it twice to prove "no duplicates"	about 15 marks
40-52	**Task 3** (PyTorch): seed, data, model, loop, plot loss. Run it	about 12 marks
52-56	Assemble ONE file (section 6), comments for every task, **run from a clean start** (Notebook: Kernel -> Restart & Run All)	catches order/state bugs
by 56	**UPLOAD.** Check the upload shows the right file name	the late penalty is steep
56-60	spare: re-upload a fix only if the upload page allows resubmission	start nothing new

Stuck for 5 minutes? Comment what you would do, leave a stub that runs, **move on** (partial code earns partial marks).

4. Two ways to produce the single .py (choose BEFORE the test)

	A. Everything in ONE Jupyter notebook	B. Tasks 1-2 in VSCode/Spyder, Task 3 in Jupyter
Good	one linear flow, nothing to paste, torch already there	normal `.py` from the start
Risk	the export can contain notebook leftovers	you must paste the Task 3 code correctly (imports + all cells)

	A. Everything in ONE Jupyter notebook	B. Tasks 1-2 in VSCode/Spyder, Task 3 in Jupyter
Must do	no `%magic` lines and no `!shell` lines (they break a `.py`); **Restart & Run All**, then export, then OPEN the exported .py and read it; rename to the required name	paste Task 3 **at the end** with its own `import torch...` lines; run the whole .py once after pasting

Either way the exported/pasted file is what is marked: **open it and run it** before uploading. Put `import torch ...` at the start of Task 3, not at the top, so Tasks 1-2 run without torch.

5. What to expect ("very similar to the 2024/25 paper" + the instructor's checklist)

- **Task 1:** load CSV, `shape`, duplicates, **merge**, filter, save CSV; scatter/line/bar/**box** plots with title + axis labels.
- **Task 2:** connect, `to_sql`, SELECT/WHERE/ORDER BY/**UPDATE** with `?`, `commit()`, **export results to CSV**.
- **Task 3: seed first**, `nn.Module` with Linear + ReLU, data `y = X @ w + b + noise` (or 2024's 10-5-3-1 sigmoid net), print shapes, loop, record + plot loss. **Do exactly what the paper says** for layers, activation, loss, optimiser.

6. The single-file layout (copy this skeleton first, 1 minute)

```
# student_name_python_task.py
# Task 1 (pandas/plots), Task 2 (SQLite), Task 3 (PyTorch)
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt

# ==================== TASK 1a: load and clean ====================
df = pd.read_csv("FILE_NAME.csv")        # exact name from the paper
print(df.shape)
# ... duplicates, merge, fill, convert, filter, save to CSV

# ==================== TASK 1b: plots ============================
# ... each plot: plt.figure(), title, xlabel, ylabel, plt.show()

# ==================== TASK 2: SQLite ============================
conn = sqlite3.connect("DB_NAME.db")     # exact DB name from the paper
cur = conn.cursor()
# ... CREATE/to_sql, SELECT, UPDATE with ?, commit(), export CSV
conn.close()

# ==================== TASK 3: PyTorch ===========================
import torch                             # imports HERE so tasks 1–2 run without torch
import torch.nn as nn
import torch.optim as optim
torch.manual_seed(42)                    # seed BEFORE data and model
# ... data, model, loss, optimiser, training loop, losses list, plot
```

7. Last 4 minutes checklist

- [ ] `.py` (NOT `.ipynb`), named `yourfirst_yourlast_python_task.py`; no `%` / `!` lines; relative paths only (`"file.csv"`)
- [ ] runs top to bottom from a clean start, no error (Notebook: Restart & Run All; or run the .py)
- [ ] a comment starts every task/sub-task in the paper's wording; every answer **printed** with a label; every plot has title + both axis labels
- [ ] shapes printed (`df.shape`, `X.shape`, `y.shape`); seed set before the data AND the model
- [ ] **uploaded before minute 56**, and the page shows the correct file name